同芯同行

Chip in, big win

UVM Scoreboard 常考题 / 真题汇总

Part 1 / 3 · 题目 01 ~ 09

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 01-09

FIFO读写验证（01）/ ALU双端口（02）/ 加法器错误统计（03）

AXI乱序响应（04）/ I2C超时检测（05）/ SPI覆盖率（06）

寄存器影子对比（07）/ PCIe顺序检测（08）/ 仲裁公平性（09）

同芯同行，真题先行

题目 01 基于 uvm_tlm_analysis_fifo 的 FIFO 读写验证

难度：初级

标签：uvm_tlm_analysis_fifo / analysis_export / get() / check_phase

【题目描述】

验证一个同步 FIFO 控制器。DUT 有独立的写入端口和读出端口，Monitor 分别采集两侧的事务并发送至 Scoreboard。Scoreboard 需要将写入侧和读出侧的数据一一配对，逐笔比对数据是否完全一致。难点在于两侧数据到达 Scoreboard 的时序不一定对齐，且仿真结束时需要确认没有任何遗留未比对的事务。本题考察 uvm_tlm_analysis_fifo 的正确使用方式以及 check_phase 的完整性验证。

【考察重点与实现要点】

> uvm_tlm_analysis_fifo 内部已实现 write()，无需在 scoreboard 中 override，只需 connect
>
> 连接方式：wr_export.connect(wr_fifo.analysis_export)，方向不能写反
>
> fifo.get() 会阻塞直到有数据，在 forever 循环中天然形成生产消费模式，无需信号量
>
> check_phase 验证 fifo.size() == 0，防止遗漏比对，是本题最常被忽略的考点
>
> run_phase 中必须 raise_objection / drop_objection，否则仿真不会等待 forever 循环

【完整答案代码】

```
// 方法论：input（写入侧）→ analysis_fifo 缓存期望值
// output（读出侧）→ imp，write() 中立即从 fifo 取值比对，无需额外队列
class fifo_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(fifo_scoreboard)

  // 写入侧：analysis_fifo 缓存期望数据（input 端）
  uvm_analysis_export #(fifo_item) wr_export;
  uvm_tlm_analysis_fifo #(fifo_item) wr_fifo;

  // 读出侧：imp 直接处理（output 端，到了立即用）
  uvm_analysis_imp #(fifo_item, fifo_scoreboard) rd_imp;

  int match_cnt, mismatch_cnt;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    wr_export = new("wr_export", this);
    wr_fifo = new("wr_fifo", this);
    rd_imp = new("rd_imp", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    wr_export.connect(wr_fifo.analysis_export);
  endfunction

  // output 端：write() 中立即从 wr_fifo 取期望值比对，不需要任何额外队列
  function void write(fifo_item rd_pkt);
    fifo_item wr_pkt;
    if (!wr_fifo.try_get(wr_pkt)) begin
      `uvm_error("SB", "读出侧数据到达，但写入侧期望队列为空！")
      return;
    end
    if (rd_pkt.data === wr_pkt.data) begin
      match_cnt++;
      `uvm_info("SB", $sformatf("MATCH data=0x%0h", rd_pkt.data), UVM_LOW)
    end else begin
      mismatch_cnt++;
      `uvm_error("SB", $sformatf("MISMATCH exp=0x%0h got=0x%0h",
                 wr_pkt.data, rd_pkt.data))
    end
  endfunction

  function void check_phase(uvm_phase phase);
    // 检查写入侧是否有未被消费的期望值
    if (wr_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "写入侧有 %0d 个事务未被读出侧消费！", wr_fifo.size()))
    `uvm_info("SB", $sformatf("总比对 = %0d，错误 = %0d",
              match_cnt + mismatch_cnt, mismatch_cnt), UVM_NONE)
  endfunction
endclass
```

【关键提示】

通过 connect_phase 把 analysis_export 连接到 fifo.analysis_export，而非直接 override write()。output 侧使用 imp，write() 中调用 try_get() 立即从 fifo 取期望值比对，无需额外队列。check_phase 验证 fifo 为空是防止遗漏比对的最后防线。

题目 02 带参考模型的 ALU 双端口 Scoreboard

难度：初级

标签：uvm_analysis_imp_decl / write_input / write_output / 内嵌参考模型

【题目描述】

验证一个 8 位 ALU，支持 ADD、SUB、AND、OR、XOR 五种运算。Monitor 同时监控 ALU 的输入总线和输出总线，并通过两个独立的 analysis_imp 端口将事务发送到 Scoreboard。Scoreboard 需要在输入侧根据操作码自行计算期望结果，并在输出侧与 DUT 实际输出进行比对。本题考察如何在同一个 Scoreboard class 中正确定义两个不同名称的 write() 函数，以及内嵌参考模型的设计方式。

【考察重点与实现要点】

> `uvm_analysis_imp_decl(_input) 和 `uvm_analysis_imp_decl(_output) 必须在 class 之前声明
>
> 两个宏分别生成 write_input() 和 write_output()，同一 class 内不能有两个 write() 重载
>
> 参考模型内嵌在 write_input() 中，避免外部依赖，保证期望值与激励同步产生
>
> case 语句覆盖所有 opcode，default 分支返回 8'hxx 并打印 warning，不能静默失败
>
> write_output() 中队列为空时必须报 uvm_error，说明时序有误而非数据错误

【完整答案代码】

```
// 方法论：input（ALU 输入）→ analysis_fifo 缓存期望值（内嵌参考模型计算后存入）
// output（ALU 输出）→ imp，write() 中立即从 fifo 取期望值比对
class alu_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(alu_scoreboard)

  // 输入侧：analysis_fifo 缓存参考模型计算的期望结果
  uvm_analysis_export #(alu_item) input_export;
  uvm_tlm_analysis_fifo #(alu_item) input_fifo;

  // 输出侧：imp，write() 中立即比对，无需额外队列
  uvm_analysis_imp #(alu_item, alu_scoreboard) output_imp;

  int match_cnt, mismatch_cnt;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    input_export = new("input_export", this);
    input_fifo = new("input_fifo", this);
    output_imp = new("output_imp", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    input_export.connect(input_fifo.analysis_export);
  endfunction

  // output 端：write() 从 fifo 取输入事务，内嵌参考模型计算期望值，立即比对
  function void write(alu_item act);
    alu_item inp;
    logic [7:0] exp;

    if (!input_fifo.try_get(inp)) begin
      `uvm_error("SB", "收到 ALU 输出但输入 fifo 为空！时序有误")
      return;
    end
```

```
    // 内嵌参考模型：在 output 的 write() 中计算期望值
    case (inp.opcode)
      ADD: exp = inp.a + inp.b;
      SUB: exp = inp.a - inp.b;
      AND: exp = inp.a & inp.b;
      OR: exp = inp.a | inp.b;
      XOR: exp = inp.a ^ inp.b;
      default: begin
        `uvm_warning("SB", $sformatf("未知 opcode=0x%0h", inp.opcode))
        exp = 8'hxx;
      end
    endcase

    if (act.result !== exp) begin
      mismatch_cnt++;
      `uvm_error("SB", $sformatf(
        "ALU 结果错误: opcode=%0d exp=0x%0h act=0x%0h",
        inp.opcode, exp, act.result))
    end else
      match_cnt++;
  endfunction

  function void check_phase(uvm_phase phase);
    if (input_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "输入 fifo 有 %0d 个事务未被消费！", input_fifo.size()))
  endfunction

  function void report_phase(uvm_phase phase);
    `uvm_info("SB", $sformatf(
      "ALU 验证完成: 通过=%0d 失败=%0d", match_cnt, mismatch_cnt), UVM_NONE)
  endfunction
endclass
```

【关键提示】

input 侧用 analysis_fifo 缓存输入事务（含操作数），output 侧用 imp，write() 中 try_get
取出输入事务，内嵌参考模型计算期望值并立即比对。这样参考模型逻辑集中在 output 的 write() 中，思路更清晰。

题目 03 带 report_phase 错误统计的加法器验证

难度：初级

标签：report_phase / check_phase / uvm_report_server / 统计汇总

【题目描述】

验证一个 8 位流水线加法器，输出包含 sum[7:0] 和 carry 进位两个字段，两者需同时验证。除逐笔功能验证外，Scoreboard 还需要在仿真结束时输出一份完整的统计报告，包含通过数量、失败数量、通过率，以及通过 UVM 全局报告服务器获取的全局 UVM 错误计数。本题考察 UVM 三个统计相关 phase 的正确分工：run_phase 做逐笔比对，check_phase 做完整性检查，report_phase 做最终汇总打印，三者职责不能混淆。

【考察重点与实现要点】

> check_phase 只做一致性检查（队列余量），不打印统计，这是与 report_phase 的分工
>
> report_phase 在 check_phase 之后运行，是打印最终统计的正确时机
>
> 通过率计算需防止除以零：total > 0 才做除法，或分母加1
>
> uvm_report_server::get_server().get_severity_count(UVM_ERROR) 获取全局错误数
>
> sum[7:0] 和 carry 分别验证，carry = exp_sum[8]，不能合并为一个条件

【完整答案代码】

```
// 方法论：output monitor 采集的事务中已包含 a、b、sum、carry 全部字段，
// 单个 imp 即可，write() 中直接内嵌参考模型计算并比对，无需队列或 run_phase。
class adder_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(adder_scoreboard)

  // 单端口：output monitor 发来的事务包含输入操作数和输出结果
  uvm_analysis_imp #(adder_item, adder_scoreboard) analysis_export;

  int unsigned pass_cnt = 0;
  int unsigned fail_cnt = 0;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
  endfunction

  // write() 中直接内嵌参考模型，立即比对，无需队列
  function void write(adder_item pkt);
    logic [8:0] exp_sum;
    exp_sum = {1'b0, pkt.a} + {1'b0, pkt.b};

    // 考点：sum[7:0] 和 carry 分开验证
    if (pkt.sum === exp_sum[7:0] && pkt.carry === exp_sum[8]) begin
      pass_cnt++;
      `uvm_info("SB", $sformatf("PASS %0d+%0d=0x%0h carry=%0b",
        pkt.a, pkt.b, pkt.sum, pkt.carry), UVM_HIGH)
    end else begin
      fail_cnt++;
      `uvm_error("SB", $sformatf(
        "FAIL %0d+%0d exp_sum=%0d exp_carry=%0b act_sum=%0d act_carry=%0b",
        pkt.a, pkt.b, exp_sum[7:0], exp_sum[8], pkt.sum, pkt.carry))
    end
  endfunction

  // 考点：check_phase 仅做完整性检查，不打统计
```

```
  function void check_phase(uvm_phase phase);
    if (pass_cnt + fail_cnt == 0)
      `uvm_warning("SB", "未收到任何事务！")
  endfunction

  // 考点：report_phase 负责最终汇总统计
  function void report_phase(uvm_phase phase);
    uvm_report_server svr = uvm_report_server::get_server();
    int total = pass_cnt + fail_cnt;
    `uvm_info("SB", $sformatf(
      "
====== Scoreboard 汇总 ======
"
      " 通过 : %0d
  失败 : %0d
"
      " 总计 : %0d 通过率 : %.1f%%
"
      " 全局 UVM 错误数 : %0d
"
      "============================",
      pass_cnt, fail_cnt, total,
      (total > 0) ? 100.0*pass_cnt/total : 0.0,
      svr.get_severity_count(UVM_ERROR)), UVM_NONE)
  endfunction
endclass
```

【关键提示】

output monitor 采集的事务中同时包含输入操作数（a、b）和输出结果（sum、carry），因此用单个 imp 即可，在 write() 中直接内嵌参考模型计算并立即比对，无需任何队列或 run_phase，是最简洁的设计。

题目 04 乱序响应比对的 AXI-Read Scoreboard

难度：中级

标签：乱序 / associative array / uvm_analysis_imp_decl / check_phase

【题目描述】

验证支持乱序响应的 AXI Read 通道。Master 依次发出若干个读请求，每个请求携带唯一的事务 ID；Slave 可以不按顺序返回响应，即先发出的请求不一定先得到响应。Scoreboard 需要对每个响应准确找到其对应的请求进行数据比对，并在仿真结束时检查是否有请求始终未收到响应。本题的核心难点在于：队列（queue）只能按顺序匹配，无法处理乱序场景，必须使用以事务 ID 为索引键的关联数组（associative array）才能正确实现。

【考察重点与实现要点】

> 乱序场景绝对不能用 queue，必须用 associative array 以事务 ID 为 key 进行索引
>
> write_rsp() 中必须先调用 exists() 再访问，否则 SystemVerilog 会自动创建空 entry 导致数据污染
>
> write_req() 中检查 ID 重复（exists 后 warning）可以早期发现 master 行为异常
>
> check_phase 检查剩余 key 是否为零，是防止 "发出但无响应" 漏检的最后保障
>
> 两个宏声明 `uvm_analysis_imp_decl(_req) 和 `uvm_analysis_imp_decl(_rsp) 必须在 class 前声明

【完整答案代码】

```
`uvm_analysis_imp_decl(_req)
`uvm_analysis_imp_decl(_rsp)

class axi_rd_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(axi_rd_scoreboard)

  uvm_analysis_imp_req #(axi_req_item, axi_rd_scoreboard) req_imp;
  uvm_analysis_imp_rsp #(axi_rsp_item, axi_rd_scoreboard) rsp_imp;

  // 考点：关联数组以事务 ID 为 key，支持乱序匹配
  logic [63:0] exp_data [int];
  int req_cnt, rsp_cnt, err_cnt;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    req_imp = new("req_imp", this);
    rsp_imp = new("rsp_imp", this);
  endfunction

  function void write_req(axi_req_item req);
    if (exp_data.exists(req.id))
      `uvm_warning("SB", $sformatf("ID=%0d 重复发送，可能覆盖上次期望值", req.id))
    exp_data[req.id] = req.exp_rdata;
    req_cnt++;
  endfunction

  function void write_rsp(axi_rsp_item rsp);
    // 考点：先 exists() 再访问，防止自动创建空 entry
    if (!exp_data.exists(rsp.id)) begin
      `uvm_error("SB", $sformatf("收到未知 ID=%0d 的响应！", rsp.id))
      err_cnt++; return;
    end
    if (rsp.rdata !== exp_data[rsp.id]) begin
      `uvm_error("SB", $sformatf(
        "ID=%0d 数据不匹配: exp=0x%0h act=0x%0h",
```

```
        rsp.id, exp_data[rsp.id], rsp.rdata))
      err_cnt++;
    end
    exp_data.delete(rsp.id); // 比对完成，释放 entry
    rsp_cnt++;
  endfunction

  // 考点：check_phase 验证无悬挂请求
  function void check_phase(uvm_phase phase);
    if (exp_data.size() != 0)
      `uvm_error("SB", $sformatf(
        "有 %0d 个请求无响应！", exp_data.size()))
    `uvm_info("SB", $sformatf(
      "AXI-RD 统计: req=%0d rsp=%0d err=%0d",
      req_cnt, rsp_cnt, err_cnt), UVM_NONE)
  endfunction
endclass
```

【关键提示】

乱序场景一定要用 associative array 而非 queue。write_rsp() 中务必先调用 exists() 再读取，否则 SV 的关联数组在访问不存在的 key 时会自动创建默认值（0），导致误报为匹配。check_phase 中检查剩余 key 数量是防止 "有去无回" 的最后一道关卡。

题目 05 带超时检测的 I2C 事务 Scoreboard

难度：中级

标签：超时 / fork-disable / event关联数组 / join_none

【题目描述】

验证 I2C 控制器的响应时序。I2C 协议要求 Slave 必须在 Master 发出命令后的 100 个时钟周期内回复响应，超时则说明总线挂起或 Slave 出现故障。Scoreboard 需要对每一笔已发出但尚未收到响应的命令独立启动一个超时计时器；当响应到达时取消对应计时器并验证数据正确性；当计时器超时时上报错误。本题考察如何在 Scoreboard 的 write() function 上下文中发起并发超时检测，以及 fork-disable 模式的正确运用。

【考察重点与实现要点】

> write_cmd() 是 function，不能调用 task，只能用 fork...join_none 启动后台超时任务
>
> automatic 关键字修饰 fork 内的局部变量，防止闭包捕获到最终值而非当前值
>
> fork-disable 模式：两分支并行，响应到达时 disable 计时分支（命名 block）
>
> event got_rsp[id] 以 id 为 key 独立区分，防止不同事务的超时互相取消
>
> 超时时报 uvm_error 而非 $fatal，允许仿真继续统计后续错误

【完整答案代码】

```
`uvm_analysis_imp_decl(_cmd)
`uvm_analysis_imp_decl(_rsp)

class i2c_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(i2c_scoreboard)

  uvm_analysis_imp_cmd #(i2c_item, i2c_scoreboard) cmd_imp;
  uvm_analysis_imp_rsp #(i2c_item, i2c_scoreboard) rsp_imp;

  i2c_item pending_q [int]; // id -> 原始命令
  event got_rsp [int]; // id -> 响应到达事件
  int TIMEOUT_CLKS = 100;
  real CLK_PERIOD = 10; // ns

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cmd_imp = new("cmd_imp", this);
    rsp_imp = new("rsp_imp", this);
  endfunction

  // 考点：write() 是 function，用 fork-join_none 启动后台任务
  function void write_cmd(i2c_item cmd);
    pending_q[cmd.id] = cmd;
    fork
      automatic int aid = cmd.id; // 考点：automatic 防闭包捕获错误值
      check_timeout(aid);
    join_none
  endfunction

  function void write_rsp(i2c_item rsp);
    if (!pending_q.exists(rsp.id)) begin
      `uvm_warning("SB", $sformatf("收到未知 id=%0d 的响应", rsp.id))
      return;
    end
    -> got_rsp[rsp.id]; // 触发事件，取消超时等待
```

```
    if (rsp.data !== pending_q[rsp.id].exp_data)
      `uvm_error("SB", $sformatf(
        "I2C 数据错误: id=%0d exp=0x%0h act=0x%0h",
        rsp.id, pending_q[rsp.id].exp_data, rsp.data))
    pending_q.delete(rsp.id);
  endfunction

  // 考点：fork-disable 超时模式
  task check_timeout(int id);
    fork
      begin : timeout_blk
        #(TIMEOUT_CLKS * CLK_PERIOD * 1ns);
        if (pending_q.exists(id))
          `uvm_error("SB", $sformatf(
            "超时！id=%0d 未在 %0d 个时钟内响应", id, TIMEOUT_CLKS))
      end
      begin : cancel_blk
        @(got_rsp[id]);
        disable timeout_blk; // 响应到达，取消超时分支
      end
    join
  endtask
endclass
```

【关键提示】

fork-disable 是 scoreboard 中处理超时的标准模式。write_cmd() 是 function，必须用 fork-join_none 启动后台 task。automatic int aid = cmd.id 是关键——若不加 automatic，fork 内部的 id 会被闭包捕获为最后一次赋值，导致所有超时任务监听同一个 ID 的事件。

题目 06 覆盖率驱动的 SPI 协议 Scoreboard

难度：初级

标签：covergroup / sample() / cross覆盖 / get_coverage() / new()实例化

【题目描述】

验证 SPI 主控制器，该控制器支持 CPOL/CPHA 四种工作模式、8/16/32 bit 三种数据长度以及多档时钟速度。除数据正确性比对之外，Scoreboard 还需要内置功能覆盖率收集，记录四种工作模式、三种数据长度和三档时钟速度各自的覆盖情况，以及工作模式与数据长度的交叉覆盖（cross coverage）。本题考察在 Scoreboard 内部正确声明并实例化 covergroup，covergroup 实例化时机，以及 write() 函数中覆盖率采样的正确时序。

【考察重点与实现要点】

> covergroup 必须在 new() 中用 = new() 实例化，若放在 build_phase 中会得到空指针
>
> write() 必须先赋值 cur_pkt = pkt 再调用 sample()，否则采样到旧数据或初始值
>
> cross 覆盖点 cx_mode_len 做 cp_mode × cp_len 的笛卡尔积，无需在 write() 中额外代码
>
> get_coverage() 返回 [0.0, 100.0] 的 real 值，在 report_phase 打印，不在 check_phase
>
> bins others = default 捕获所有未被显式命名的枚举值，防止未知值被遗漏

【完整答案代码】

```
// 方法论：input（SPI 发送）→ analysis_fifo 缓存期望值
// output（SPI 接收）→ imp，write() 中采样覆盖率后立即从 fifo 取期望值比对
class spi_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(spi_scoreboard)

  // 输入侧：analysis_fifo 缓存发送事务（含期望 rx_data）
  uvm_analysis_export #(spi_item) input_export;
  uvm_tlm_analysis_fifo #(spi_item) input_fifo;

  // 输出侧：imp，write() 立即处理
  uvm_analysis_imp #(spi_item, spi_scoreboard) output_imp;

  spi_item cur_pkt; // covergroup 通过此成员变量采样

  // 考点：covergroup 在 class 内声明，引用成员变量
  covergroup spi_cg;
    cp_mode: coverpoint cur_pkt.cpol_cpha {
      bins mode0 = {2'b00}; // CPOL=0 CPHA=0
      bins mode1 = {2'b01}; // CPOL=0 CPHA=1
      bins mode2 = {2'b10}; // CPOL=1 CPHA=0
      bins mode3 = {2'b11}; // CPOL=1 CPHA=1
    }
    cp_len: coverpoint cur_pkt.data_len {
      bins len_8 = {8};
      bins len_16 = {16};
      bins len_32 = {32};
      bins others = default;
    }
    cp_speed: coverpoint cur_pkt.clk_div {
      bins slow = {[1:4]};
      bins medium = {[5:16]};
      bins fast = {[17:$]};
    }
    // 考点：cross 交叉覆盖，无需额外采样代码
    cx_mode_len: cross cp_mode, cp_len;
```

```
  endgroup

  // 考点：covergroup 必须在 new() 中实例化，不能在 build_phase
  function new(string name, uvm_component parent);
    super.new(name, parent);
    spi_cg = new();
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    input_export = new("input_export", this);
    input_fifo = new("input_fifo", this);
    output_imp = new("output_imp", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    input_export.connect(input_fifo.analysis_export);
  endfunction

  // output 端：write() 中先采样覆盖率，再从 fifo 取输入事务立即比对
  function void write(spi_item act);
    spi_item inp;

    // 考点：先赋值 cur_pkt 再 sample，顺序不能反
    cur_pkt = act;
    spi_cg.sample();

    if (!input_fifo.try_get(inp)) begin
      `uvm_error("SB", "收到 SPI 输出但输入 fifo 为空！")
      return;
    end

    if (act.rx_data !== inp.exp_rx)
      `uvm_error("SB", $sformatf(
        "SPI 数据不匹配: mode=%0b exp=0x%0h act=0x%0h",
```

```
        inp.cpol_cpha, inp.exp_rx, act.rx_data))
  endfunction

  function void check_phase(uvm_phase phase);
    if (input_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "输入 fifo 有 %0d 个事务未被消费！", input_fifo.size()))
  endfunction

  // 考点：report_phase 打印覆盖率
  function void report_phase(uvm_phase phase);
    `uvm_info("SB", $sformatf(
      "SPI 功能覆盖率: %.2f%%", spi_cg.get_coverage()), UVM_NONE)
  endfunction
endclass
```

【关键提示】

> input 侧（发送帧含期望 rx_data）用 analysis_fifo，output 侧（接收结果）用 imp。write() 中先采样覆盖率（先赋值 cur_pkt 再 sample），再 try_get 取发送帧立即比对，无需 pkt_q 和 run_phase。

题目 07 基于 uvm_reg 的寄存器影子对比 Scoreboard

难度：高级

标签：uvm_reg / get_reg_by_offset / predict / get_mirrored_value

【题目描述】

验证一个包含寄存器文件的 APB 外设。Scoreboard 监听 APB 总线上的所有读写事务，内部维护一份与 DUT 寄存器文件对应的软件影子模型。每次写操作时更新影子寄存器的预测值，每次读操作时将 DUT 实际读回的数据与影子值进行比对，自动检测寄存器读写错误。本题考察 UVM 寄存器模型的三个核心 API：get_reg_by_offset()（通过地址查找寄存器对象）、predict()（更新影子值）、get_mirrored_value()（获取当前影子期望值）。

【考察重点与实现要点】

> get_reg_by_offset() 返回 null 时必须判断，否则后续调用会崩溃
>
> predict() 的 .kind 参数：UVM_PREDICT_WRITE 用于写操作更新影子值
>
> get_mirrored_value() 返回软件影子预测值，是 scoreboard 比对的基准
>
> uvm_config_db 获取失败时要 `uvm_fatal，而非 warning 后继续运行
>
> 地址未命中（null）用 uvm_warning，因为可能是合理的未映射地址空间

【完整答案代码】

```
class reg_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(reg_scoreboard)

  uvm_analysis_imp #(apb_item, reg_scoreboard) analysis_export;
  my_reg_block reg_blk;
  int match_cnt, mismatch_cnt;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
    // 考点：从 config_db 获取寄存器模型句柄
    if (!uvm_config_db #(my_reg_block)::get(this, "", "reg_model", reg_blk))
      `uvm_fatal("SB", "无法获取寄存器模型 reg_model，请确认已在 test 中设置")
  endfunction

  function void write(apb_item pkt);
    uvm_reg target_reg;
    uvm_reg_data_t exp_val, act_val;

    // 考点：通过地址查找寄存器对象
    target_reg = reg_blk.default_map.get_reg_by_offset(pkt.addr);
    if (target_reg == null) begin
      `uvm_warning("SB", $sformatf(
        "地址 0x%0h 无对应寄存器，跳过", pkt.addr))
      return;
    end

    if (pkt.pwrite) begin
      // 写操作：用 predict() 更新影子寄存器
      void'(target_reg.predict(pkt.pwdata, .kind(UVM_PREDICT_WRITE)));
      `uvm_info("SB", $sformatf(
        "写 %s = 0x%0h", target_reg.get_name(), pkt.pwdata), UVM_HIGH)
    end else begin
      // 读操作：影子值与 DUT 实际读回值比对
      exp_val = target_reg.get_mirrored_value();
```

```
      act_val = pkt.prdata;
      if (act_val !== exp_val) begin
        mismatch_cnt++;
        `uvm_error("SB", $sformatf(
          "寄存器 %s 读取不匹配: exp=0x%0h act=0x%0h",
          target_reg.get_name(), exp_val, act_val))
      end else
        match_cnt++;
    end
  endfunction

  function void report_phase(uvm_phase phase);
    `uvm_info("SB", $sformatf(
      "寄存器比对: 通过=%0d 失败=%0d", match_cnt, mismatch_cnt), UVM_NONE)
  endfunction
endclass
```

【关键提示】

get_reg_by_offset() 是核心 API，返回 null 时必须检查后跳过，否则后续调用会产生空指针崩溃。predict() 的 .kind(UVM_PREDICT_WRITE) 参数更新影子值；get_mirrored_value()
读取影子值用于比对。两者缺一不可，混用会导致影子值与比对基准不一致。

题目 08 事务重排序检测 Scoreboard（有序协议验证）

难度：中级

标签：序列号连续性 / PCIe TLP / CRC校验 / 乱序检测

【题目描述】

验证要求严格顺序输出的 PCIe TLP（事务层数据包）处理逻辑。每个 TLP 携带一个单调递增的序列号和一个 CRC 校验字段。DUT 必须按序列号顺序输出 TLP，任何跳号（乱序）或序列号回退（重复、倒退）都应被检测并上报；同时每个 TLP 的 CRC 字段需要独立校验。本题考察两个相互独立的验证维度（顺序性检测和 CRC 校验）如何在 write() 这一 function 上下文中同时实现，以及出现乱序后如何保证后续比对不连锁失败。

【考察重点与实现要点】

> write() 是 function 不是 task，不能加 delay，任何需要等待的逻辑要在 run_phase 中完成
> 无论发现乱序与否，都必须更新 exp_seq = pkt.seq_num + 1，否则后续全部误报
> delta > 0 是跳号（中间事务丢失），delta < 0 是回退或重复，需分开报不同错误类型
> $abs(delta) 计算乱序量绝对值，max_reorder 统计最大乱序幅度，用于评估乱序严重程度
> CRC 校验与序列号检测是独立的两个验证维度，不能因序列号错误就跳过 CRC 检查

【完整答案代码】

```
class ordering_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(ordering_scoreboard)

  uvm_analysis_imp #(tlp_item, ordering_scoreboard) analysis_export;

  int unsigned exp_seq = 0;
  int unsigned order_err = 0;
  int unsigned max_reorder = 0;
  int unsigned total_tlp = 0;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
  endfunction

  // 考点：write() 是 function，绝对不能加 delay/wait
  function void write(tlp_item pkt);
    int signed delta;
    total_tlp++;

    // 检查序列号连续性
    if (pkt.seq_num != exp_seq) begin
      delta = int'(pkt.seq_num) - int'(exp_seq);
      order_err++;
      if ($abs(delta) > int'(max_reorder))
        max_reorder = $abs(delta);

      if (delta > 0)
        `uvm_error("SB", $sformatf(
          "TLP 乱序: 期望 seq=%0d 实际=%0d（跳过 %0d 个）",
          exp_seq, pkt.seq_num, delta))
      else
        `uvm_error("SB", $sformatf(
          "TLP 回退/重复: 期望 seq=%0d 实际=%0d",
          exp_seq, pkt.seq_num))
```

```
    end

    // 考点：独立验证 CRC（不受序列号错误影响）
    if (pkt.crc !== calc_crc(pkt.payload))
      `uvm_error("SB", $sformatf(
        "TLP CRC 校验失败: seq=%0d", pkt.seq_num))

    // 考点：无论乱序与否都必须更新期望序号
    exp_seq = pkt.seq_num + 1;
  endfunction

  function int calc_crc(logic [7:0] data[]);
    int crc = 0;
    foreach (data[i]) crc ^= int'(data[i]);
    return crc;
  endfunction

  function void report_phase(uvm_phase phase);
    `uvm_info("SB", $sformatf(
      "TLP 顺序验证: 总数=%0d 乱序错误=%0d 最大乱序量=%0d",
      total_tlp, order_err, max_reorder), UVM_NONE)
  endfunction
endclass
```

【关键提示】

write() 是 function，所有超时/等待逻辑必须放在 run_phase 中用 fork 实现。即使发现乱序也必须更新 exp_seq，否则后续所有事务都会误报。CRC
和序列号是两个独立的验证维度，任何一个发现错误都不应跳过另一个的检查。

题目 09 多通道仲裁 Scoreboard（公平性验证）

难度：高级

标签：统计验证 / $countones / 关联数组统计 / report_phase

【题目描述】

验证一个 4 通道加权轮询仲裁器。仲裁器每个时钟周期以单热码（one-hot）形式输出 grant_oh[3:0]，指示当前被授权的通道。各通道的期望授权权重可通过 uvm_config_db 进行配置，默认为均等分配（各 25%）。Scoreboard 需要长时间运行并统计各通道实际获得授权的次数，在仿真结束时验证各通道的授权比例是否与配置权重一致（允许 ±5% 的偏差范围）。本题考察统计性验证的方法，以及单热码合法性检查（功能验证）与比例统计验证的配合使用。

【考察重点与实现要点】

> $countones(grant_oh) > 1 说明仲裁器错误地同时授权多个通道，是功能级错误
> 统计验证必须在 report_phase 做，check_phase 时仿真刚结束，总数可能不足以统计
> $clog2() 只对精确的2的幂次有效，通用方法是 foreach 遍历每一 bit 找授权通道
> 偏差计算：actual_pct - exp_weight[i]，可正可负，用 $abs() 与 tolerance 比较
> 从 config_db 读取权重允许不同测试用例复用同一 scoreboard，只需修改配置即可

【完整答案代码】

```
class arbiter_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(arbiter_scoreboard)

  uvm_analysis_imp #(arb_item, arbiter_scoreboard) analysis_export;

  int unsigned grant_cnt[4]; // 各通道累计授权次数
  int unsigned total_grants = 0;
  real tolerance = 5.0; // 允许 ±5% 偏差
  real exp_weight[4] = '{25.0, 25.0, 25.0, 25.0}; // 期望权重（%）

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
    // 从配置读取各通道权重
    void'(uvm_config_db #(real)::get(this, "", "ch0_weight", exp_weight[0]));
    void'(uvm_config_db #(real)::get(this, "", "ch1_weight", exp_weight[1]));
    void'(uvm_config_db #(real)::get(this, "", "ch2_weight", exp_weight[2]));
    void'(uvm_config_db #(real)::get(this, "", "ch3_weight", exp_weight[3]));
  endfunction

  function void write(arb_item pkt);
    int ch;
    // 考点：$countones 检查单热码合法性（功能验证）
    if ($countones(pkt.grant_oh) > 1)
      `uvm_error("SB", "仲裁器同时授权多个通道！")
    if (pkt.grant_oh == '0) return; // 本周期无授权，跳过

    // 考点：遍历 bit 找授权通道（比 $clog2 更通用）
    ch = -1;
    for (int i = 0; i < 4; i++)
      if (pkt.grant_oh[i]) ch = i;

    if (ch >= 0) begin
      grant_cnt[ch]++;
      total_grants++;
```

```
    end
  endfunction

  // 考点：统计验证必须在 report_phase 做
  function void report_phase(uvm_phase phase);
    real actual_pct, deviation;
    `uvm_info("SB", "
=== 仲裁公平性报告 ===", UVM_NONE)
    if (total_grants == 0) begin
      `uvm_warning("SB", "未收到任何 grant 事务！") return;
    end
    for (int i = 0; i < 4; i++) begin
      actual_pct = 100.0 * real'(grant_cnt[i]) / real'(total_grants);
      deviation = actual_pct - exp_weight[i];
      if ($abs(deviation) > tolerance)
        `uvm_error("SB", $sformatf(
          "通道 %0d 公平性超限: 期望 %.1f%% 实际 %.1f%% 偏差 %.1f%%",
          i, exp_weight[i], actual_pct, deviation))
      else
        `uvm_info("SB", $sformatf(
          "通道 %0d: 期望 %.1f%% 实际 %.1f%% 偏差 %.1f%% OK",
          i, exp_weight[i], actual_pct, deviation), UVM_NONE)
    end
  endfunction
endclass
```

【关键提示】

统计验证与功能验证是两个不同层次：$countones 检查单热码是功能验证（每拍只能一个授权），比例偏差检查是统计验证（长期分布是否符合权重）。统计验证必须在 report_phase
做，因为需要所有事务处理完后才能计算出有意义的比例。